

UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

Programa de Ingeniería de Sistemas y Computación

INFORME DE INVESTIGACIÓN

Taller segundo corte: Docker

Arquitectura de Software

Thiffany Torres Aparicio

Cod. T00082951

Luis Giovanny Carreno Ortiz

Cartagena de Indias, Colombia

Octubre de 2026

INTRODUCCIÓN

Docker es una plataforma que permite construir, distribuir y ejecutar aplicaciones dentro de contenedores. En desarrollo de software ayuda a mantener entornos más consistentes entre computadores y a separar los componentes de un sistema. El objetivo de este informe es explicar qué es Docker, cuáles son sus componentes, sus funcionalidades y comandos principales, el problema que resuelve y su importancia dentro de la arquitectura de software.

¿QUÉ ES DOCKER?

Docker permite empaquetar una aplicación y los elementos que necesita para ejecutarse. Un contenedor es un proceso aislado que utiliza el kernel del sistema operativo anfitrión, pero mantiene separados determinados recursos y archivos. Por eso suele ser más ligero que una máquina virtual tradicional, que normalmente incluye un sistema operativo invitado completo. Esto no significa que un contenedor no necesite medidas de seguridad: sus permisos y configuración siguen siendo importantes.

Por ejemplo, una API puede requerir una versión concreta de Go y una base de datos MySQL. En vez de instalar manualmente todos los componentes en cada computador, el proyecto puede definirlos y ejecutarlos mediante contenedores.

COMPONENTES PRINCIPALES

- **CLI (cliente Docker):** herramienta de terminal que recibe comandos como docker ps y se comunica con el motor de Docker.
- **Daemon o motor (dockerd):** servicio que administra imágenes, contenedores, redes y volúmenes.
- **Imagen:** plantilla que contiene los archivos y metadatos para crear contenedores.
- **Contenedor:** instancia ejecutable creada a partir de una imagen.
- **Dockerfile:** archivo con instrucciones para construir una imagen.
- **Registro:** servicio como Docker Hub, desde el cual se descargan o al cual se publican imágenes.

Cadena general: Dockerfile → construcción de imagen → creación e inicio del contenedor. El registro participa al descargar imágenes base o al distribuir imágenes creadas.

FUNCIONALIDADES PRINCIPALES

Portabilidad: la misma imagen puede ejecutarse en distintos equipos con Docker compatible. Por ejemplo, un compañero puede ejecutar la API sin instalar manualmente todas sus dependencias.

Aislamiento: los servicios pueden ejecutarse por separado. Por ejemplo, la API y MySQL pueden tener contenedores distintos.

Reproducibilidad: los archivos de construcción y configuración reducen diferencias entre entornos, aunque también hay que controlar variables y versiones.

Versionado: etiquetas como mysql:8 identifican versiones o variantes de imágenes.

Escalabilidad: se pueden ejecutar varias instancias de un servicio cuando la aplicación y la infraestructura lo permiten.

CI/CD: los procesos automatizados pueden construir, probar y publicar imágenes durante la integración y entrega continua.

COMANDOS PRINCIPALES

Comando	¿Qué hace?	Ejemplo
Docker pull	Descarga una imagen	Docker pull mysql:8
Docker run	Crea e inicia un contenedor	Docker run –name prueba nginx
Docker ps	Lista contenedores activos	Docker ps
Docker ps -a	Lista todos los contenedores	Docker ps -a
Docker logs	Muestra los registros	Docker logs mi -api
Docker exec	Ejecuta un comando dentro de un contenedor activo	Docker exec -it mi-api sh
Docker stop/start	Detiene o inicia un contenedor	Docker stop mi-api
Docker rm	Elimina un contenedor detenido	Docker rm mi-api
Docker images	Lista imágenes locales	Docker images
Docker rmi	Elimina una imagen local	Docker rmi nginx
Docker build	Construye una imagen desde un dockerfile	Docker build -t mi -api.
Docker compose up	Crea e inicia servicios definido en compose	Docker compose up -- build

PARÁMETROS DE TRES COMANDOS

Docker run: -d ejecuta en segundo plano; -p publica un puerto; -e define variables de entorno; -v monta un volumen; y --name asigna un nombre.

Ejemplo:

docker run -d --name web -p 8080:80 nginx

Docker build: -t asigna un nombre y etiqueta; el punto final indica el contexto de construcción.

Ejemplo:

docker build -t biblioteca-api .

Docker Compose up: crea o inicia los servicios descritos en el archivo Compose. --build solicita construir las imágenes antes de iniciar.

Ejemplo:

docker compose up –build

¿QUÉ PROBLEMA RESUELVE DOCKER?

El problema conocido como «en mi máquina funciona» ocurre cuando una aplicación funciona en un computador, pero falla en otro por diferencias en versiones de dependencias, sistema operativo, variables o pasos manuales de instalación. Por ejemplo, una API puede esperar una versión concreta de una librería o credenciales distintas para conectarse a la base de datos.

Antes: cada integrante instala y configura los componentes por separado, por lo que pueden aparecer diferencias.

Después: el proyecto define imágenes y servicios configurados, lo que hace más reproducible el entorno. Docker reduce este tipo de problemas, aunque no elimina todos los errores.

IMPORTANCIA EN LA ARQUITECTURA DE SOFTWARE

Docker permite separar una API, una base de datos o un broker de eventos en contenedores distintos. Esto facilita administrar, probar y actualizar cada servicio, siempre que se configuren correctamente las redes, los volúmenes, las variables y los permisos.

En microservicios, cada servicio puede empaquetarse y desplegarse de forma independiente. En arquitecturas SOA y orientadas a eventos, Docker puede ejecutar servicios y brokers que forman parte del sistema. Docker Compose ayuda a describir y levantar varios servicios relacionados en desarrollo y pruebas.

Las imágenes permiten mantener una base común entre desarrollo, pruebas y producción, con la configuración apropiada para cada entorno. Docker también facilita desplegar aplicaciones en servidores o plataformas de nube compatibles. Para sistemas grandes pueden requerirse herramientas adicionales de orquestación, monitoreo y seguridad.

CONCLUSIÓN

Con esta investigación comprendí que Docker permite empaquetar y ejecutar aplicaciones en contenedores, ayuda a mantener entornos consistentes y separa los componentes de un sistema. Sus imágenes, Dockerfiles y comandos facilitan la construcción y ejecución, mientras que Compose coordina servicios relacionados. Como estudiante de Ingeniería de Sistemas, considero que Docker es útil para desarrollar, probar y desplegar proyectos de forma más organizada.

REFERENCIAS

Docker. (s. f.). *Docker documentation*. https://docs.docker.com/

Docker. (s. f.). *Docker Engine*. https://docs.docker.com/engine/

Docker. (s. f.). *Docker Compose*. https://docs.docker.com/compose/

Docker. (s. f.). *Dockerfile reference*. https://docs.docker.com/reference/dockerfile/